MERMAID 11.14 · LATTICE THEME

Every Mermaid diagram, one slide each.

A reference gallery of the 26 diagram types Mermaid supports, rendered with the Lattice palette so you can see what each one looks like in production.

H

ORIENTATION · HOW TO USE THIS DECK

How this gallery is organised.

Mermaid 11.14 · rendered with Lattice theme

THREE GROUPS, TWENTY-SIX DIAGRAMS

Stable diagrams render with full theme control. Experimental ones marked 🔥 may evolve in syntax and have limited theming surfaces.

15

Stable

Flowchart, sequence, class, state, ER, journey, gantt, pie, quadrant, requirement, gitgraph, c4, mindmap, timeline, zenuml.

11

Experimental 🔥

Sankey, xy chart, block, packet, kanban, architecture, radar, treemap, venn, ishikawa, treeView.

6

Themeable

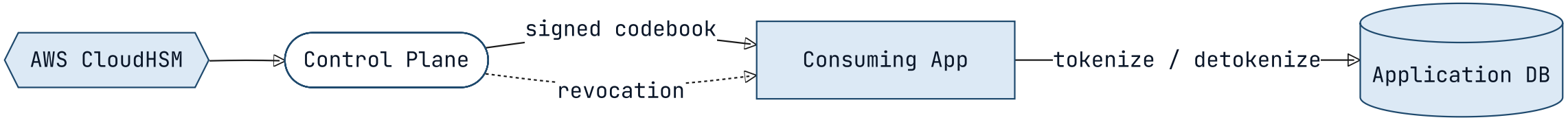
Flowchart, sequence, pie, state, class, journey have named themeVariables. Everything else inherits from the core six.

GROUP 01 · STABLE DIAGRAMS

The fifteen well-supported types.

01 · FLOWCHART

Boxes and arrows. The most-used diagram type.



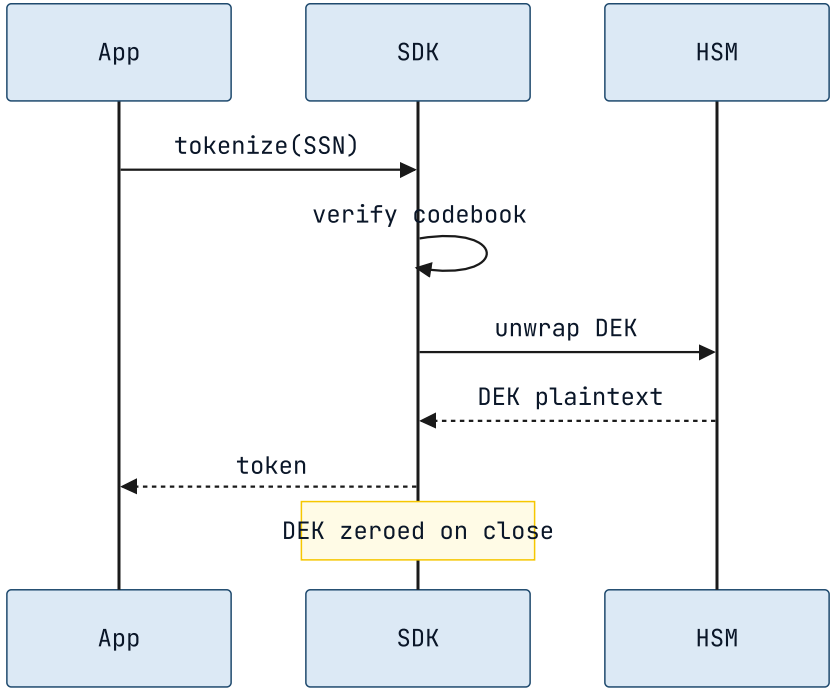
KEY INSIGHT

Full theme support — node fills, borders, edge colors, cluster fills, edge labels all controllable via themeVariables.

02 · SEQUENCE DIAGRAM

Actors and signals over time.

Tokenize call sequence

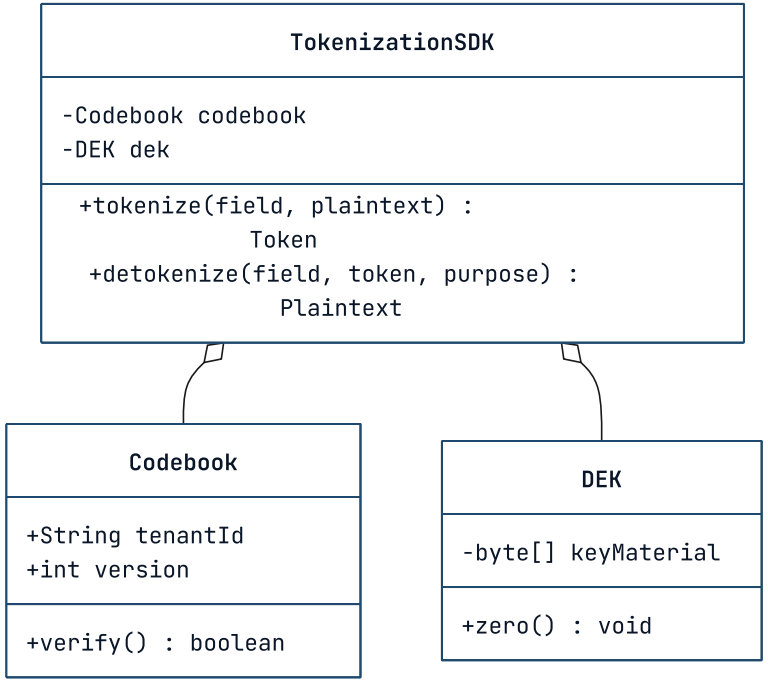


KEY INSIGHT

Full theme support — actor backgrounds, signal colors, activation bars, note styling all themeable.

03 · CLASS DIAGRAM

UML classes with attributes, methods, and inheritance.

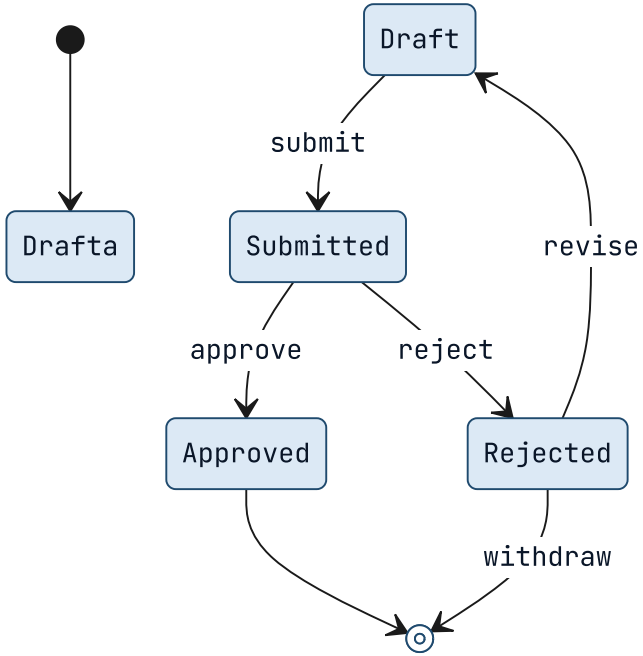


KEY INSIGHT

Themeable — `classText` controls label color; node fills inherit from `primaryColor`.

04 · STATE DIAGRAM

A finite state machine.

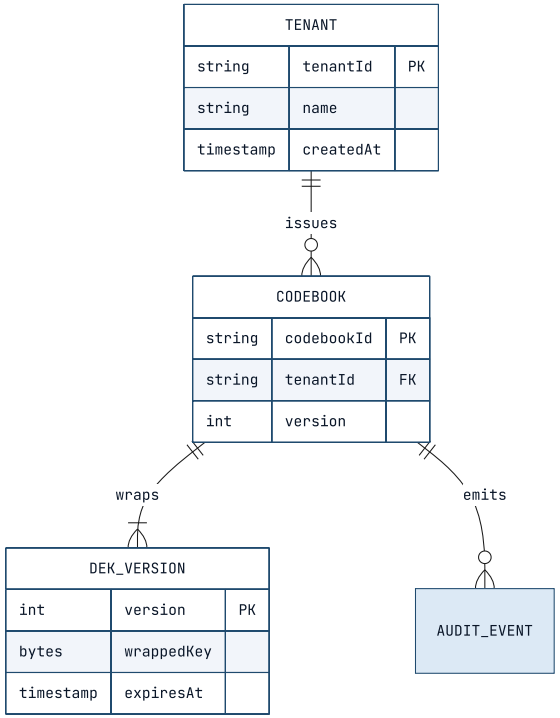


KEY INSIGHT

Themeable — `labelColor` and `altBackground` for composite states. State fills inherit from primary.

05 · ENTITY RELATIONSHIP DIAGRAM

Database tables and their relationships.

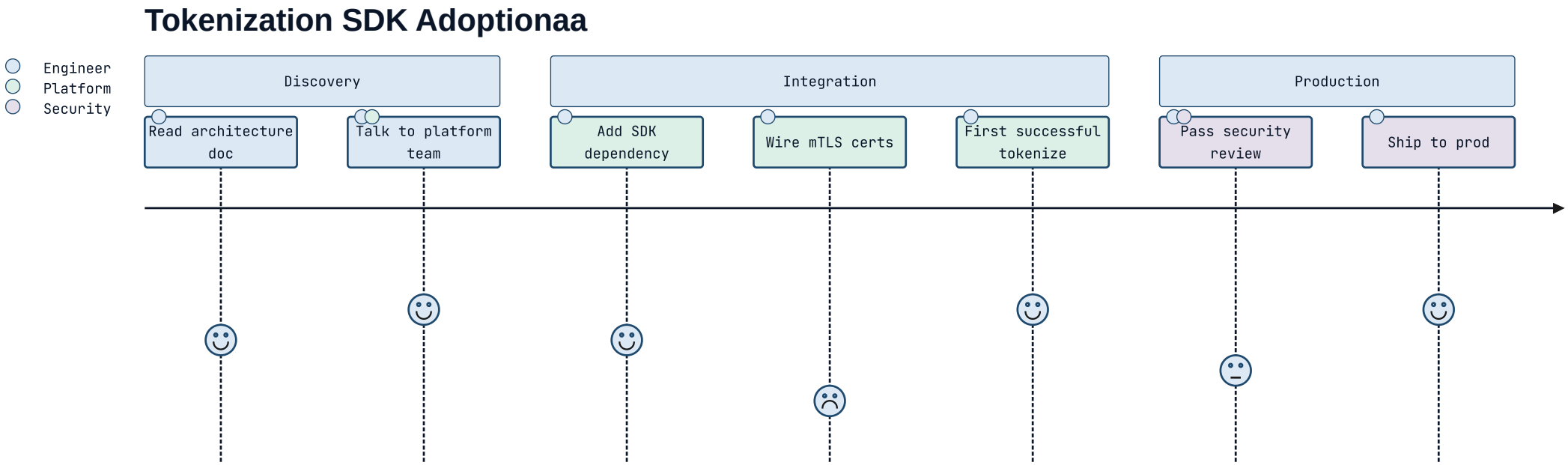


KEY INSIGHT

No named themeVariables — inherits border / text / fill from core variables.

06 · USER JOURNEY

Steps and the emotional state at each one.

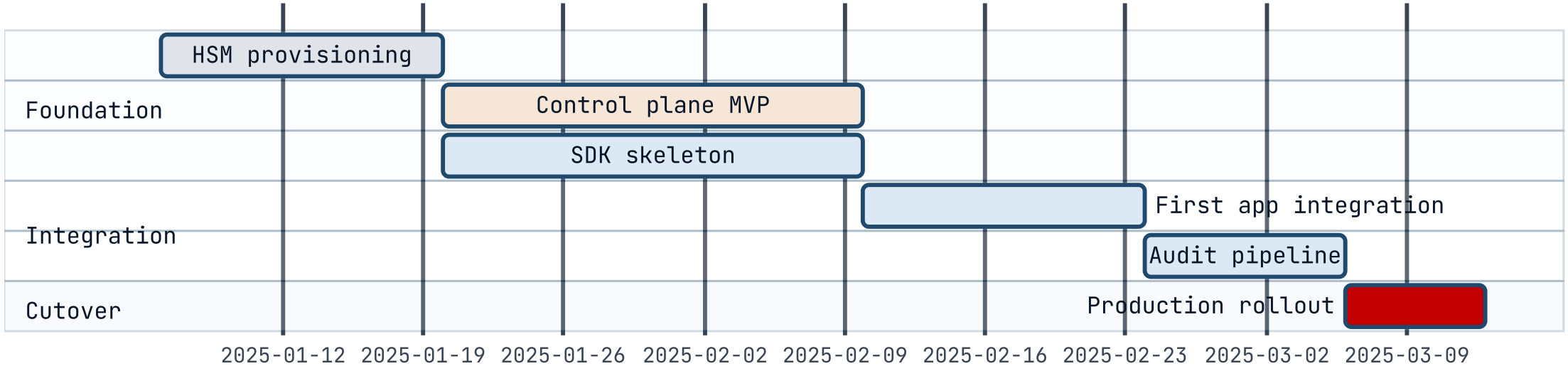


KEY INSIGHT

Themeable — eight `fillType0..7` variables for the section coloring.

07 · GANTT

Tasks across a timeline, with dependencies.

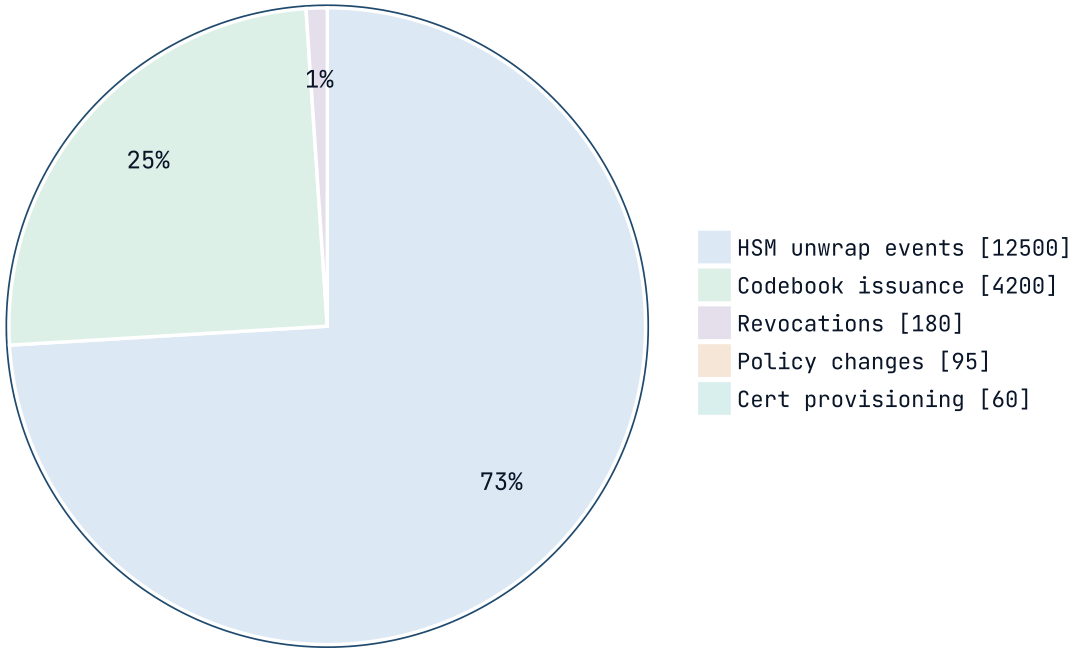


KEY INSIGHT

Lots of dedicated themeVariables — task bar, active task, done task, critical, today line all separately controllable.

08 · PIE CHART

Proportions of a whole.



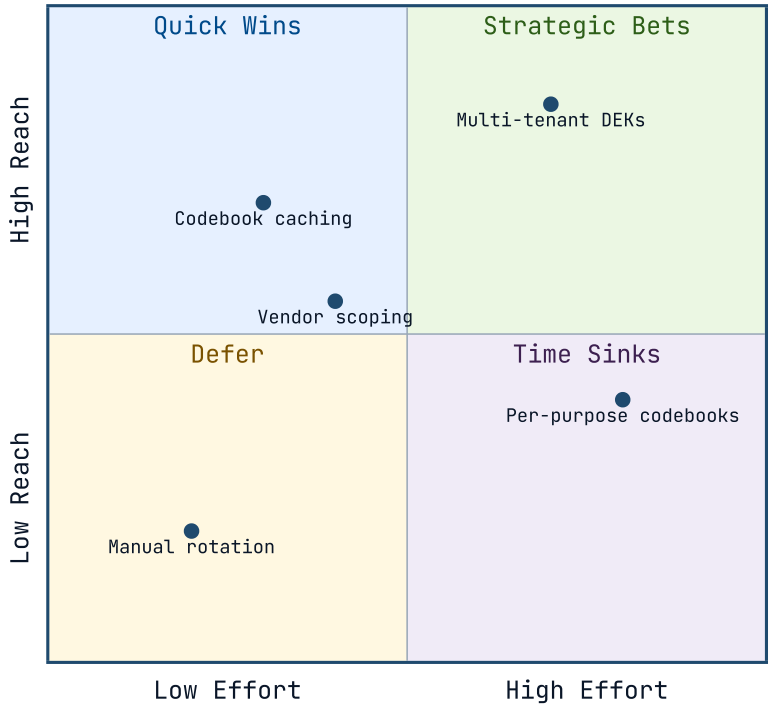
KEY INSIGHT

Most theme-rich diagram — twelve `pie1..12` slot colors plus title, section, legend, stroke variables.

09 · QUADRANT CHART

Two-axis priority scatter.

Reach vs effort

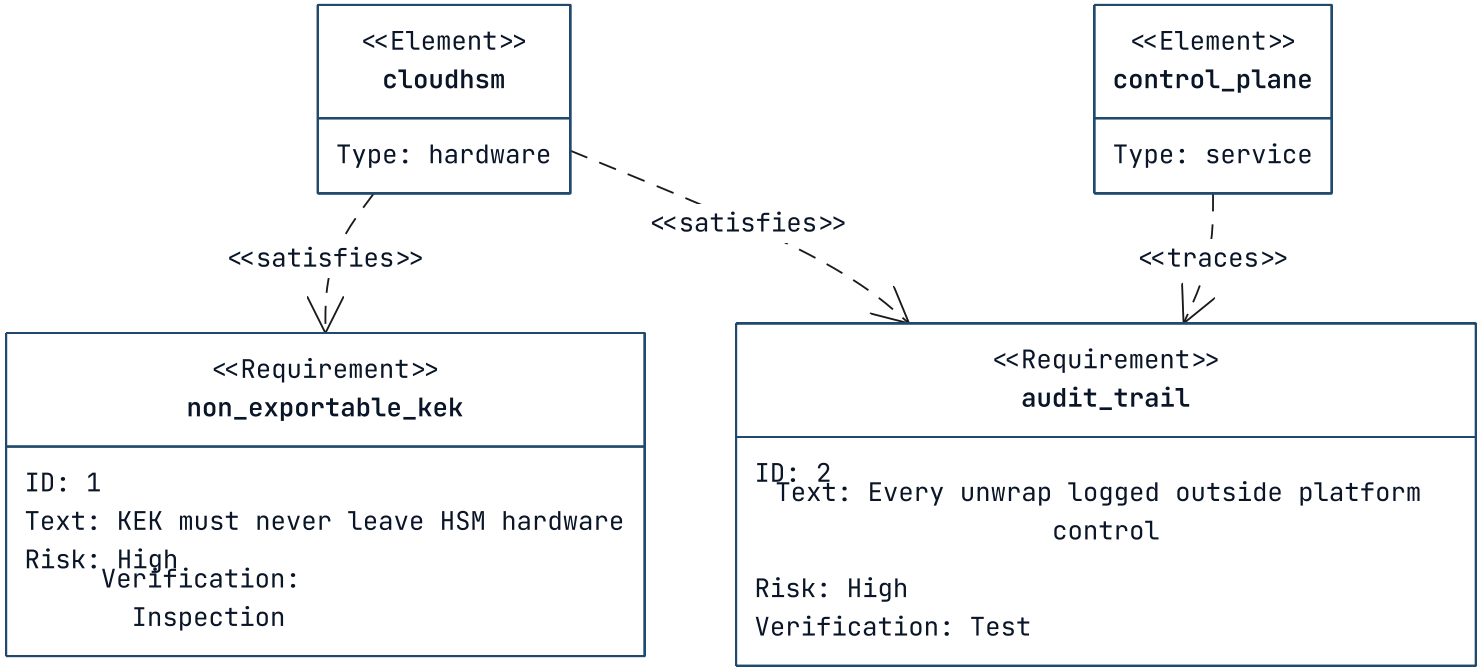


KEY INSIGHT

Themeable — four quadrant fills and text colors, axis labels, point fill all controllable.

10 · REQUIREMENT DIAGRAM

SysML-style requirements with traceability.

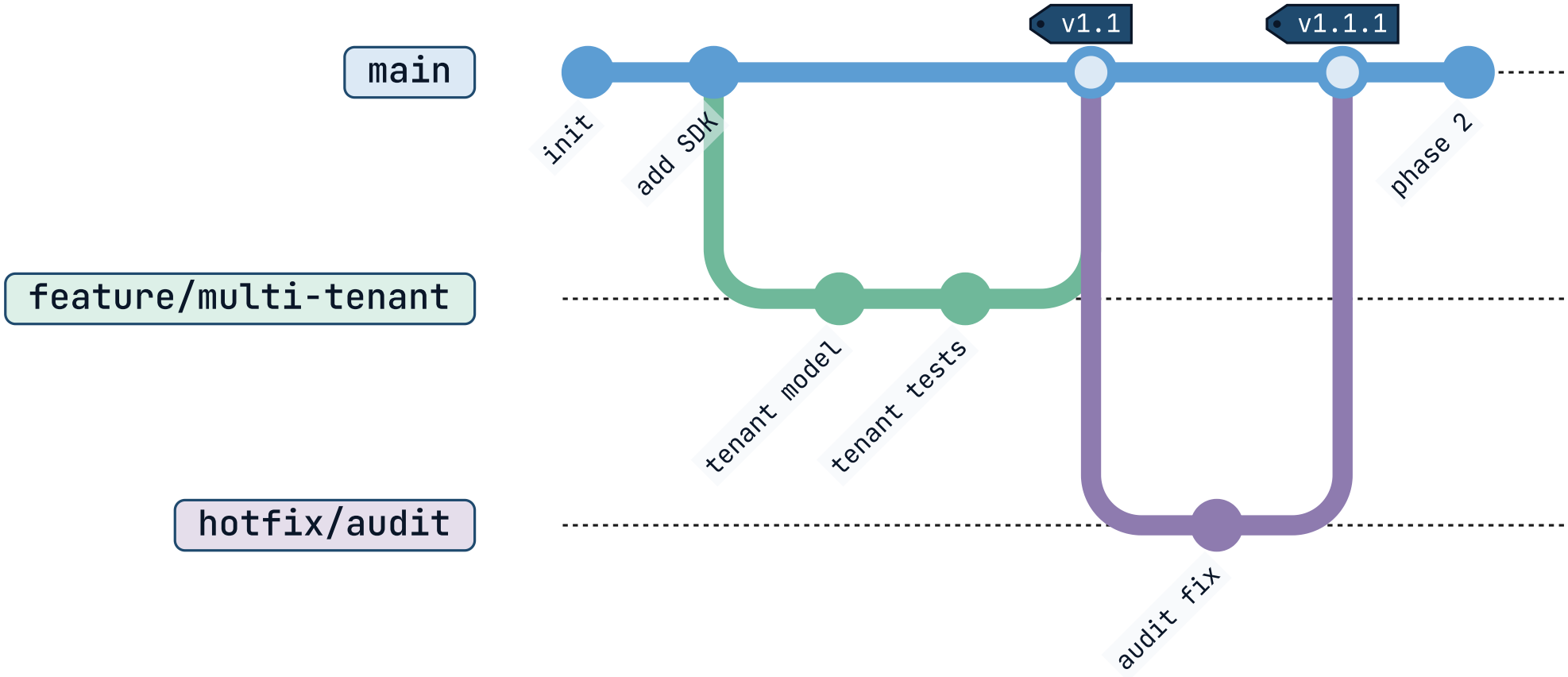


KEY INSIGHT

Inherits from core variables. No diagram-specific theme keys.

11 · GITGRAPH

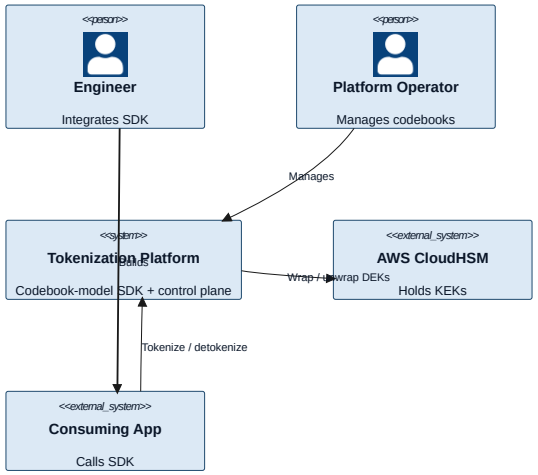
Branching, merging, tags.



12 · C4 DIAGRAM

Software architecture at four zoom levels.

Tokenization Platform · System Context

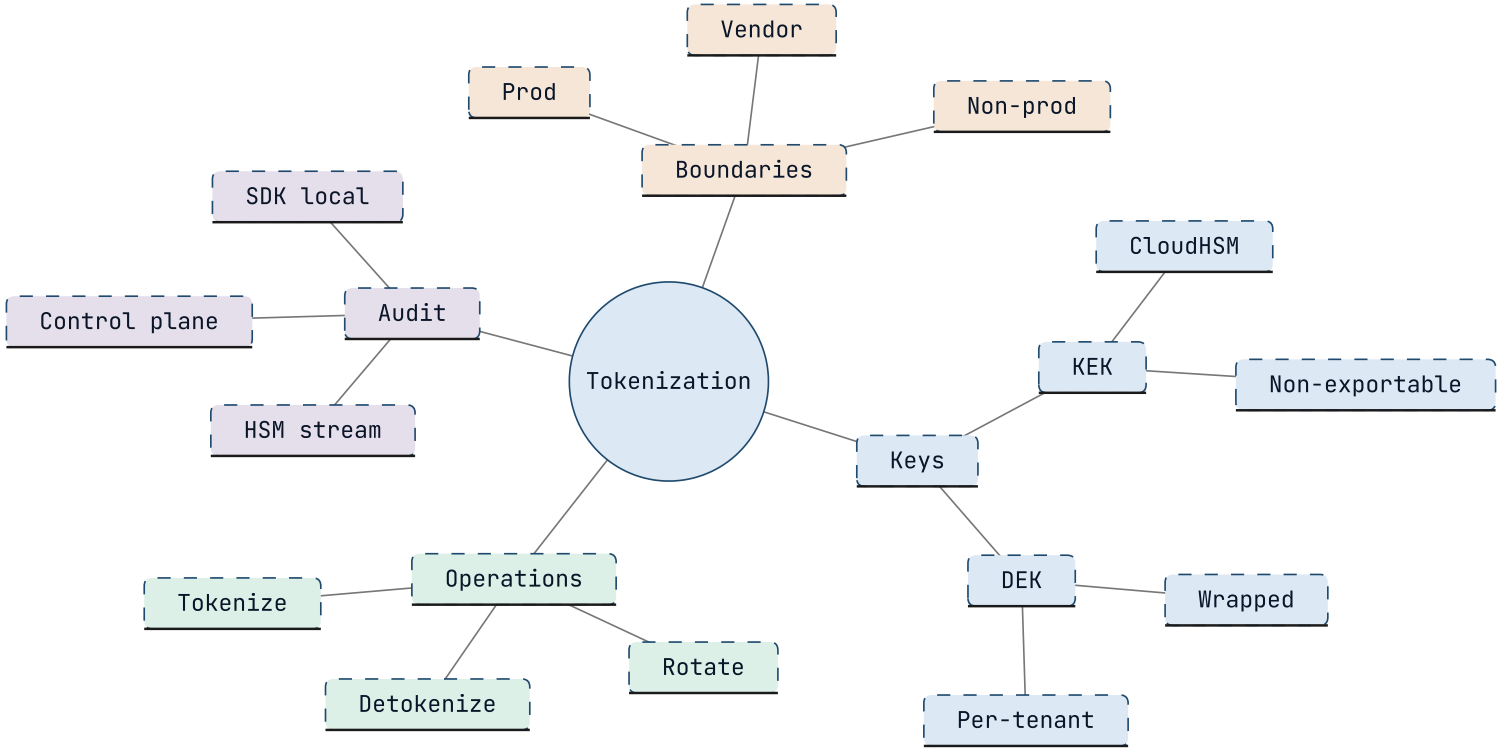


KEY INSIGHT

Marked 🦺⚠️ in the docs — supported but the team flags it as work-in-progress. Theme support partial.

13 · MINDMAP

Hierarchical brainstorm.

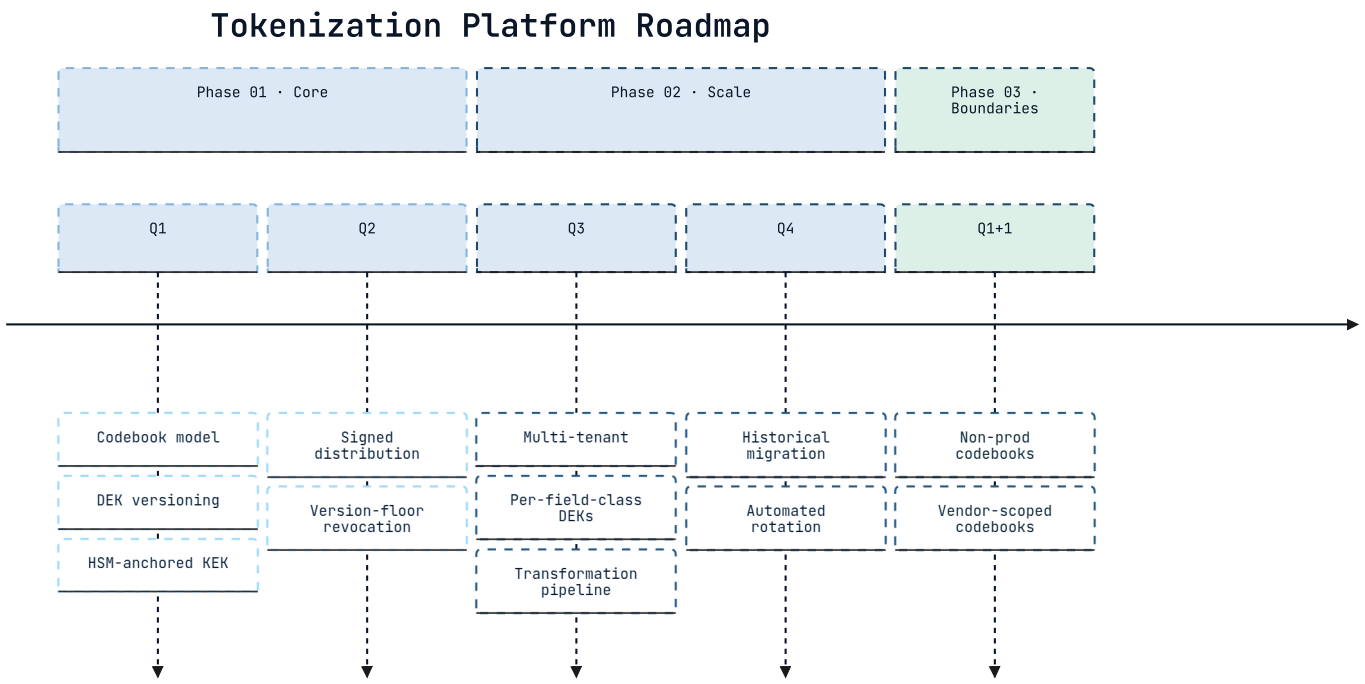


KEY INSIGHT

Inherits from core. Multiple shape syntaxes: `((round))`, `[square]`, `(rounded)`, `))cloud((`, `)hex(`.

14 · TIMELINE

Events on a horizontal axis.



KEY INSIGHT

Inherits from core. The section coloring uses the `cScale` palette.

15 · ZENUML

A simpler sequence-diagram dialect.

ZenUML is a Mermaid diagram type that emits HTML+SVG with Tailwind CSS classes for styling. The Mermaid CLI can produce the markup, but it does not bundle the @zenuml/core stylesheet — so static SVG export renders the structure without the visual styling. ZenUML works in browser contexts where the Tailwind utilities resolve.

For static-PDF pipelines like this one, prefer the standard `sequenceDiagram` (slide 5) — same conceptual model, fully supported by the Mermaid CLI, themable through `themeVariables`.

```
zenuml
  title Order
  Customer→Order.create() {
    Order→Inventory.check() {
      return available
    }
    return confirmed
  }
```

KEY INSIGHT

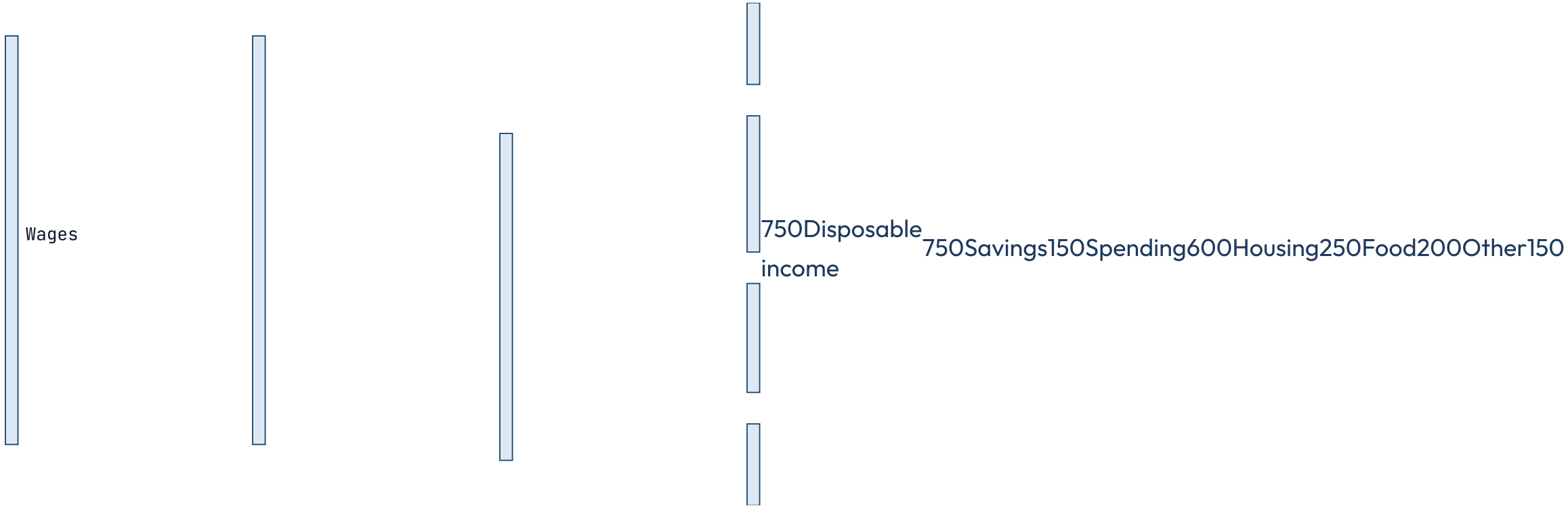
ZenUML uses HTML/Tailwind rendering rather than pure SVG. Use sequenceDiagram for static export contexts.

GROUP 02 · EXPERIMENTAL DIAGRAMS

The eleven new types marked 🔥.

16 · SANKEY 🔥

Flow volumes between nodes.

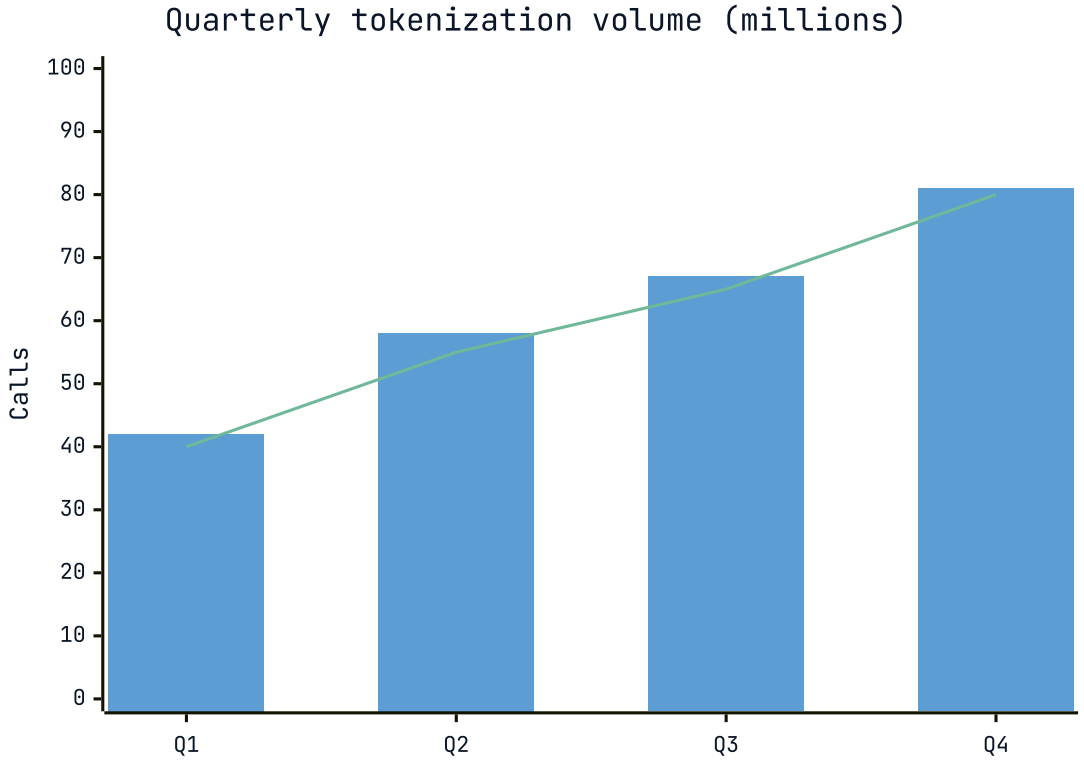


KEY INSIGHT

CSV-style input. Theme inheritance only — link colors are computed from node colors via the `linkColor` config.

17 · XY CHART 🔥

Bar and line chart on a numeric axis.

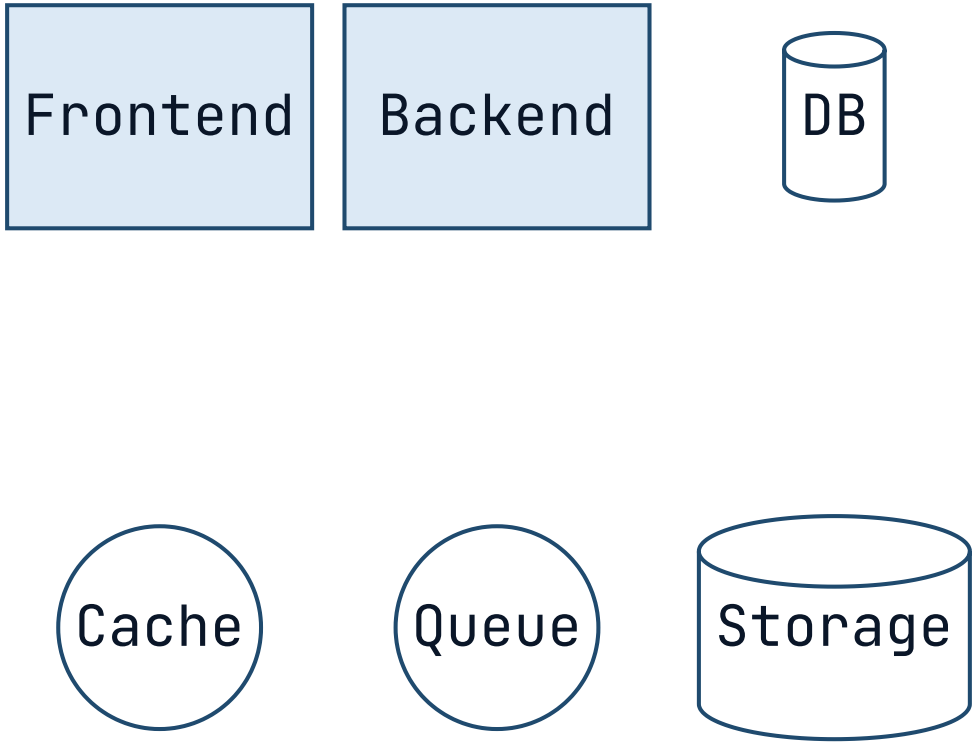


KEY INSIGHT

Inherits from core. The plot color palette can be overridden but per-series control is limited.

18 · BLOCK DIAGRAM 🔥

Tile layout for system blocks.

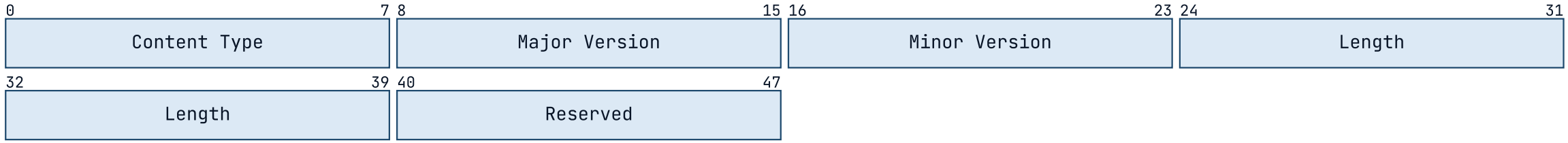


KEY INSIGHT

Inherits from core. Spacing controlled by the `space:N` directive.

19 · PACKET 🔥

Bit-level packet layout.

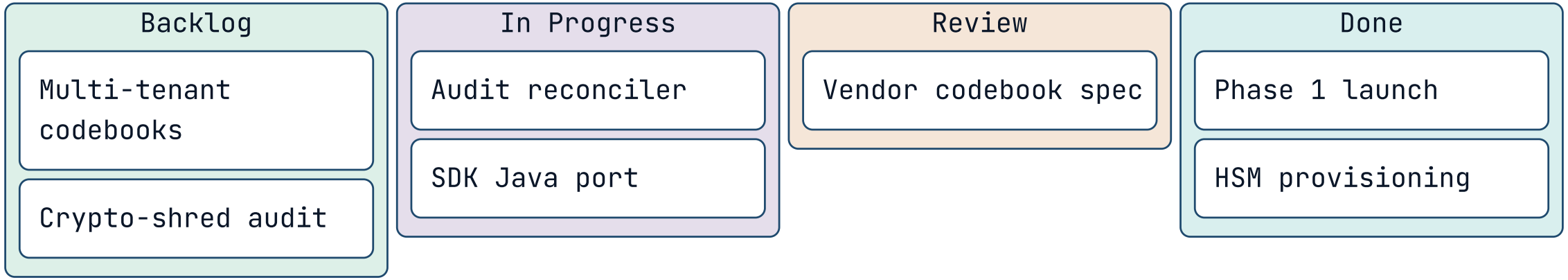


KEY INSIGHT

Inherits from core. Width-fixed at 32 bits per row by default; `0-N` ranges define each field.

20 · KANBAN 🔥

A board with swim-lane columns.

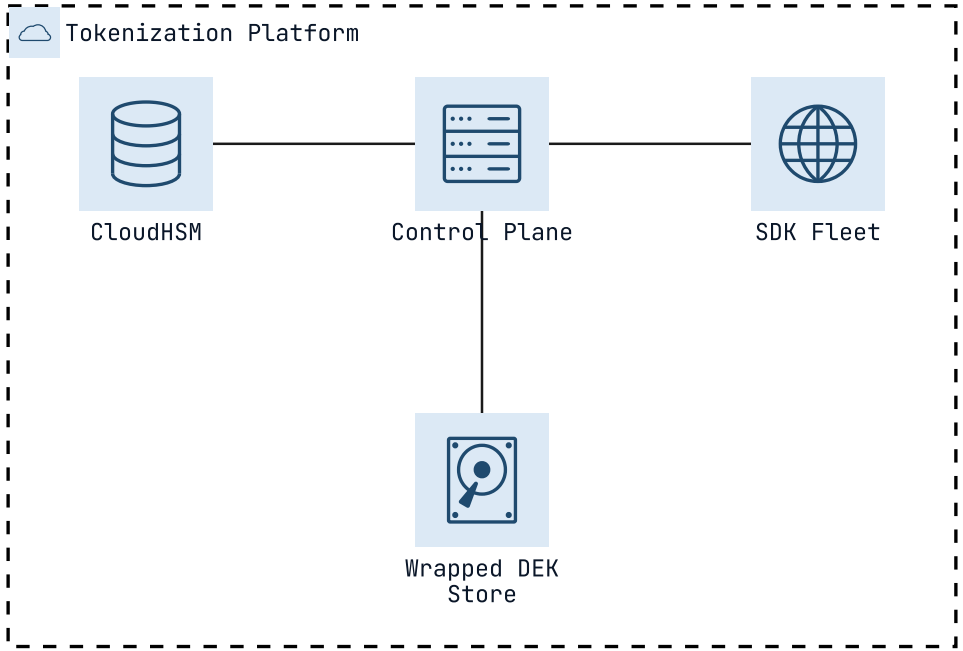


KEY INSIGHT

Inherits from core. Per-card `[Title]@{ assigned: ..., priority: ... }` metadata supported but optional.

21 · ARCHITECTURE 🔥

Cloud-architecture style with services and groups.

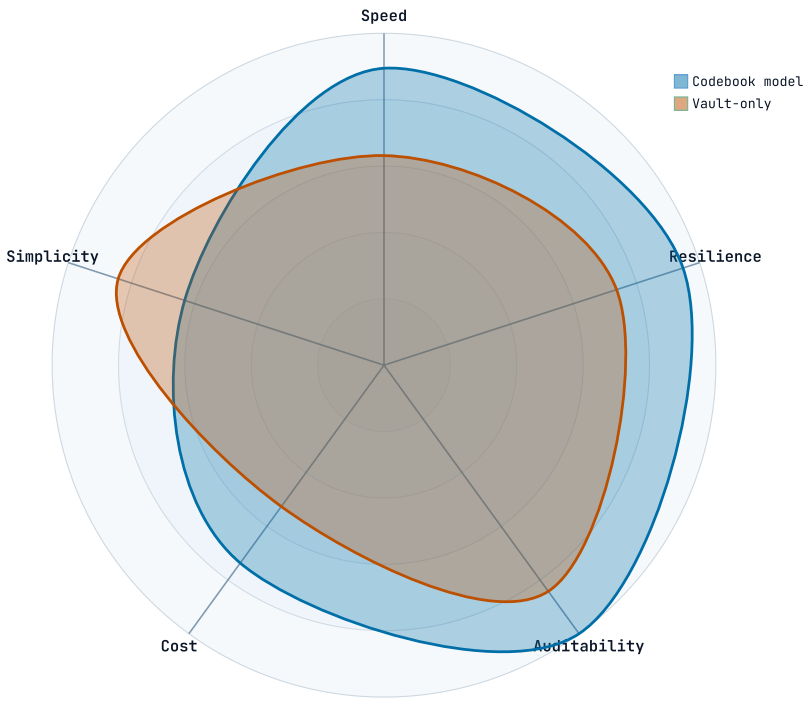


KEY INSIGHT

Inherits from core. Icons drawn from a registered set; default set is limited — register more via `mermaid.registerIconPacks`.

22 · RADAR 🔥

Multi-axis polar chart.

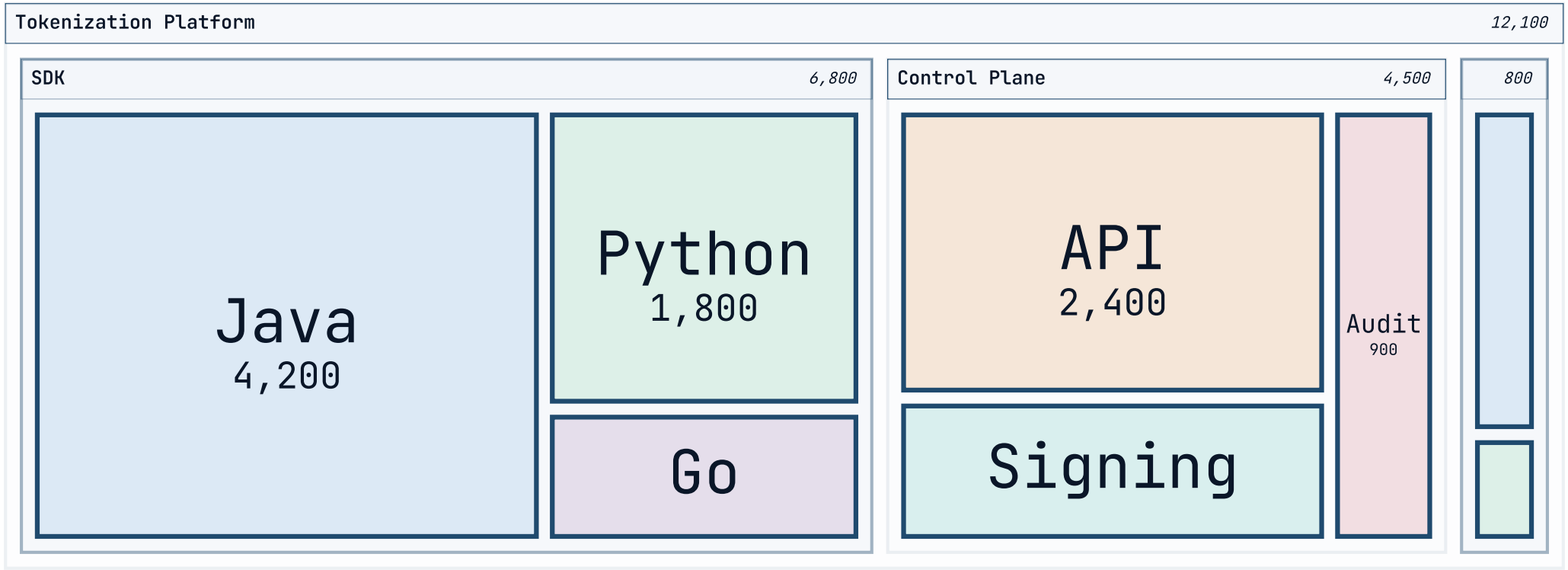


KEY INSIGHT

Inherits from core. Multiple `curve` lines plot on the same axes.

23 · TREEMAP 🔥

Nested rectangles sized by value.

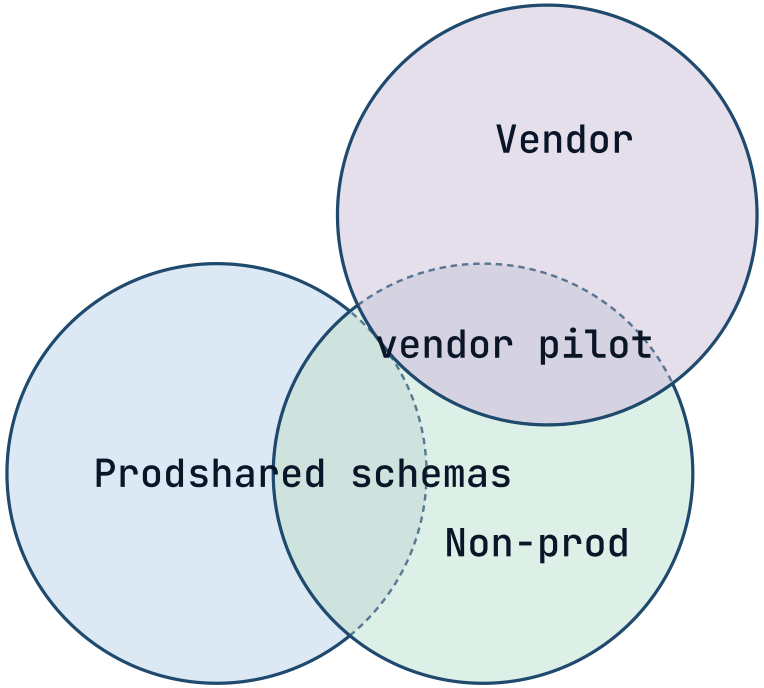


KEY INSIGHT

Inherits from core. Sized values cascade from leaves up; intermediate nodes don't take a value.

24 · VENN 🔥

Set overlap.

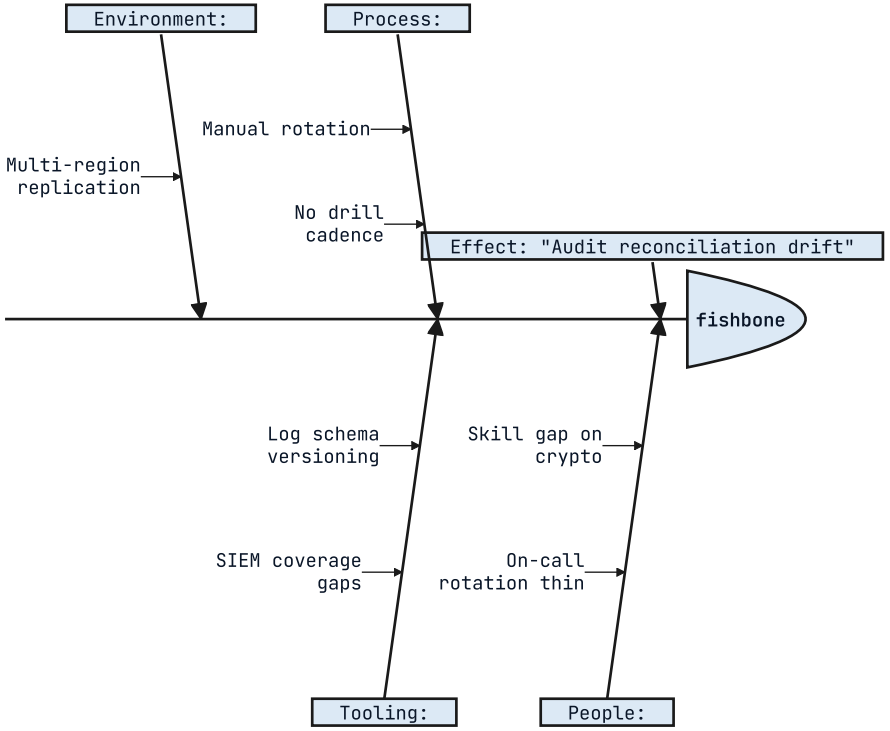


KEY INSIGHT

Inherits from core. `union A, B` for two-set overlap; `set X["Label"]` to give a display name distinct from the identifier; `union A, B["..."]` to label an intersection region.

25 · ISHIKAWA 🔥

Fishbone cause-and-effect.



KEY INSIGHT

Inherits from core. Branches default to four (people, process, tooling, environment) but any heading is allowed.

26 · TREEVIEW 🔥

Indented file-tree style hierarchy.

```
– /
  ├ src/
  │ ├ main.js
  │ └ lib/
  │   ├ tokenize.js
  │   └ verify.js
  └ package.json
```

KEY INSIGHT

Three named theme variables: `labelFontSize`, `labelColor`, `lineColor`. Indentation alone determines the tree.

MERMAID 11.14 · LATTICE THEME

Twenty-six diagrams, one theme.

Stable diagrams have rich theme control. Experimental diagrams marked 🔥 inherit from the core six variables and may evolve in syntax. Use this gallery as a reference for which diagram fits a given problem.